Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-212БВ-25

Студент: Минин А.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 19.10.26

Москва, 2026

Постановка задачи

Вариант 11.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Unix). Ограничение максимального количества потоков, работающих в один момент времени, задаётся ключом запуска программы. Необходимо уметь продемонстрировать количество потоков, используемое программой, с помощью стандартных средств операционной системы. В отчёте привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков и объяснить полученные результаты. Задание варианта: наложить K раз медианный фильтр на матрицу, состоящую из целых чисел. Размер окна задаётся пользователем.

Общий метод и алгоритм решения

Использованные системные вызовы:

- pthread_create(&thread, NULL, func, arg); – создание рабочего потока, выполняющего функцию filter_thread над своей полосой строк матрицы;
- pthread_join(thread, NULL); – ожидание завершения рабочего потока;
- pthread_barrier_init / pthread_barrier_wait / pthread_barrier_destroy – барьер, на котором потоки синхронизируются после каждой итерации фильтра;
- clock_gettime(CLOCK_MONOTONIC, &ts); – измерение времени по «настенным» часам, только вокруг фильтрации (без чтения и записи файлов);

Программа получает пять аргументов командной строки: имя входного файла, имя выходного файла, число потоков, число итераций K и размер окна фильтра. Входной файл содержит в первой строке число строк и столбцов, далее элементы матрицы. Алгоритм работы:

1. Разбор аргументов. Число потоков и размер окна проверяются на корректность. Размер окна должен быть положительным нечётным числом: окно строится как $(2 \cdot r+1)^2$ элементов, где r = размер/2.
2. Чтение матрицы из файла и выделение второй матрицы того же размера. Две матрицы используются как пара буферов: на каждой итерации одна читается, другая записывается, затем они меняются ролями.
3. Медианный фильтр для одного элемента. Для каждого элемента собирается окно из соседних значений; элементы за границей матрицы считаются равными 0. Окно сортируется (qsort), медианой становится средний элемент отсортированного окна.
4. Однопоточный режим (число потоков равно 1). K раз вызывается функция matrix_filter_sequential, после каждого вызова буферы меняются местами.
5. Многопоточный режим. Число потоков ограничивается числом строк матрицы. Каждый поток получает равное количество строк, плюс первые n потоков получают оставшиеся n строк, каждый по одной.
6. Синхронизация. На итерации k+1 в окно попадают соседние строки, посчитанные другими потоками, поэтому после каждой итерации все потоки ждут друг друга на барьере.
7. Выбор результата. Потоки меняют только свои копии указателей, поэтому матрицы в main остаются на месте: после чётного числа итераций результат лежит во входной матрице, после нечётного – в выходной.
8. Замер времени выполняется в main вокруг фильтрации и выводится в stderr в миллисекундах.

Код программы

lab2.c

```
#include <stdint.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
#include <time.h>

typedef struct {
    size_t rows;
    size_t cols;
    int *data;
} Matrix;

typedef struct {
    Matrix *input;
    Matrix *output;

    size_t filter_size;
    size_t iterations;

    size_t first_row;
    size_t last_row;

    int *window;

    pthread_barrier_t *barrier;
} ThreadArgs;

static Matrix matrix_alloc(size_t rows, size_t cols)
{
    Matrix m;
    m.rows = rows;
    m.cols = cols;
    if (cols != 0 && rows > SIZE_MAX / cols / sizeof(int)) {
        fprintf(stderr, "matrix size overflow\n");
        exit(EXIT_FAILURE);
    }
    m.data = malloc(rows * cols * sizeof(int));
    if (!m.data) {
        fprintf(stderr, "malloc failed\n");
        exit(EXIT_FAILURE);
    }
    return m;
}

static void matrix_free(Matrix *m)
{
    m->cols = 0;
    m->rows = 0;
    free(m->data);
    m->data = NULL;
}
```

```
static Matrix matrix_read(FILE *file)
{
    if (file == NULL) {
        fprintf(stderr, "NULL path\n");
        exit(EXIT_FAILURE);
    }
    size_t rows;
    size_t cols;

    if (fscanf(file, "%zu %zu", &rows, &cols) != 2) {
        fprintf(stderr, "failed to read matrix dimensions\n");
        exit(EXIT_FAILURE);
    }

    Matrix matrix = matrix_alloc(rows, cols);

    for (size_t i = 0; i < rows * cols; ++i) {
        if (fscanf(file, "%d", &matrix.data[i]) != 1) {
            fprintf(stderr, "failed to read matrix element\n");
            matrix_free(&matrix);
            exit(EXIT_FAILURE);
        }
    }

    return matrix;
}

static int compare_ints(const void *a, const void *b)
{
    int x = *(const int *)a;
    int y = *(const int *)b;
    return (x > y) - (x < y);
}

static int is_out_of_bounds(const Matrix *matrix, long row, long col)
{
    return (row < 0 ||
            row >= (long)matrix->rows ||
            col < 0 ||
            col >= (long)matrix->cols);
}

static void apply_median_filter_row(
    const Matrix *input,
    Matrix *output,
    size_t row,
    size_t filter_size,
    int *window)
{
    size_t radius = filter_size / 2;
    size_t window_size = filter_size * filter_size;

    for (size_t col = 0; col < input->cols; ++col) {
        size_t count = 0;

        for (long di = -(long)radius; di <= (long)radius; ++di) {
```

```
            long input_row = (long)row + di;

            for (long dj = -(long)radius; dj <= (long)radius; ++dj) {
                long input_col = (long)col + dj;

                if (is_out_of_bounds(input, input_row, input_col)) {
                    window[count++] = 0;
                } else {
                    window[count++] = input->data[input_row * input->cols + input_col];
                }
            }
        }

        qsort(window, window_size, sizeof *window, compare_ints);

        output->data[row * output->cols + col] = window[window_size / 2];
    }
}

static void matrix_filter_sequential(
    const Matrix *input,
    Matrix *output,
    size_t filter_size)
{
    size_t window_size = filter_size * filter_size;

    int *window = malloc(window_size * sizeof *window);
    if (window == NULL) {
        fprintf(stderr, "malloc failed\n");
        exit(EXIT_FAILURE);
    }

    for (size_t row = 0; row < input->rows; ++row) {
        apply_median_filter_row(input, output, row, filter_size, window);
    }

    free(window);
}

static void *filter_thread(void *arg)
{
    ThreadArgs *args = arg;

    for (size_t k = 0; k < args->iterations; ++k) {
        for (size_t row = args->first_row; row < args->last_row; ++row) {
            apply_median_filter_row(
                args->input,
                args->output,
                row,
                args->filter_size,
                args->window
            );
        }

        pthread_barrier_wait(args->barrier);
```

```
        Matrix *temp = args->input;
        args->input = args->output;
        args->output = temp;
    }

    return NULL;
}

static void matrix_filter_parallel(
    Matrix *input,
    Matrix *output,
    size_t filter_size,
    size_t iterations,
    size_t thread_count)
{
    if (thread_count > input->rows) {
        thread_count = input->rows;
    }
    if (thread_count == 0) {
        return;
    }

    pthread_t *threads = malloc(thread_count * sizeof *threads);
    ThreadArgs *args = malloc(thread_count * sizeof *args);

    if (threads == NULL || args == NULL) {
        fprintf(stderr, "malloc failed\n");
        exit(EXIT_FAILURE);
    }

    pthread_barrier_t barrier;

    if (pthread_barrier_init(&barrier, NULL, (unsigned)thread_count) != 0) {
        fprintf(stderr, "failed to initialize barrier\n");
        exit(EXIT_FAILURE);
    }

    size_t base_rows = input->rows / thread_count;
    size_t extra_rows = input->rows % thread_count;

    size_t window_size = filter_size * filter_size;

    for (size_t i = 0; i < thread_count; ++i) {
        args[i].input = input;
        args[i].output = output;

        args[i].filter_size = filter_size;
        args[i].iterations = iterations;

        args[i].first_row = i * base_rows + (i < extra_rows ? i : extra_rows);
        args[i].last_row = args[i].first_row + base_rows + (i < extra_rows ? 1 : 0);

        args[i].window = malloc(window_size * sizeof *args[i].window);

        if (args[i].window == NULL) {
            fprintf(stderr, "windows malloc failed\n");
```

```
            exit(EXIT_FAILURE);
        }

        args[i].barrier = &barrier;

        if (pthread_create(&threads[i], NULL, filter_thread, &args[i]) != 0) {
            fprintf(stderr, "failed to create thread %zu\n", i);
            exit(EXIT_FAILURE);
        }
    }

    for (size_t i = 0; i < thread_count; ++i) {
        pthread_join(threads[i], NULL);
    }

    pthread_barrier_destroy(&barrier);

    for (size_t i = 0; i < thread_count; ++i) {
        free(args[i].window);
    }

    free(args);
    free(threads);
}

static void matrix_write(FILE *file, const Matrix *matrix)
{
    fprintf(file, "%zu %zu\n", matrix->rows, matrix->cols);

    for (size_t i = 0; i < matrix->rows; ++i) {
        for (size_t j = 0; j < matrix->cols; ++j) {
            fprintf(file, "%d", matrix->data[i * matrix->cols + j]);

            if (j + 1 < matrix->cols) {
                fputc(' ', file);
            }
        }

        fputc('\n', file);
    }
}

int main(int argc, char *argv[])
{
    if (argc != 6) {
        fprintf(
            stderr,
            "Usage: %s <input_file> <output_file> "
            "<thread_count> <iterations> <filter_size>\n",
            argv[0]
        );
        return EXIT_FAILURE;
    }

    const char *input_filename = argv[1];
    const char *output_filename = argv[2];
```

```
char *endptr;

unsigned long thread_count_value = strtoul(argv[3], &endptr, 10);

if (*argv[3] == '\0' ||
    *endptr != '\0' ||
    thread_count_value == 0) {

    fprintf(stderr, "invalid thread count\n");
    return EXIT_FAILURE;
}

unsigned long iterations_value = strtoul(argv[4], &endptr, 10);

if (*argv[4] == '\0' ||
    *endptr != '\0') {

    fprintf(stderr, "invalid iterations count\n");
    return EXIT_FAILURE;
}

unsigned long filter_size_value = strtoul(argv[5], &endptr, 10);

if (*argv[5] == '\0' ||
    *endptr != '\0' ||
    filter_size_value == 0 ||
    filter_size_value % 2 == 0 ||
    filter_size_value > SIZE_MAX / filter_size_value / sizeof(int)) {

    fprintf(stderr, "invalid filter size (must be a positive odd number)\n");
    return EXIT_FAILURE;
}

size_t thread_count = (size_t)thread_count_value;
size_t iterations = (size_t)iterations_value;
size_t filter_size = (size_t)filter_size_value;

FILE *input_file = fopen(input_filename, "r");
if (input_file == NULL) {
    perror("failed to open input file");
    return EXIT_FAILURE;
}

Matrix input = matrix_read(input_file);
fclose(input_file);

Matrix output = matrix_alloc(input.rows, input.cols);

const Matrix *result;

struct timespec t0, t1;
clock_gettime(CLOCK_MONOTONIC, &t0);

if (thread_count == 1) {
    for (size_t k = 0; k < iterations; ++k) {
```

```
            matrix_filter_sequential(&input, &output, filter_size);
            Matrix tmp = input;
            input = output;
            output = tmp;
        }
        result = &input;
    } else {
        matrix_filter_parallel(&input, &output, filter_size, iterations, thread_count);
        result = (iterations % 2 == 0) ? &input : &output;
    }

    clock_gettime(CLOCK_MONOTONIC, &t1);
    double ms = (t1.tv_sec - t0.tv_sec) * 1000.0 + (t1.tv_nsec - t0.tv_nsec) / 1e6;
    printf("%.3f ms\n", ms);

    FILE *output_file = fopen(output_filename, "w");
    if (output_file == NULL) {
        perror("failed to open output file");

        matrix_free(&input);
        matrix_free(&output);

        return EXIT_FAILURE;
    }

    matrix_write(output_file, result);

    fclose(output_file);

    matrix_free(&input);
    matrix_free(&output);

    return EXIT_SUCCESS;
}
```

Протокол работы программы

Запустим программу с параметрами: input.txt – файл на вход, output.txt – файл на выход, 4 – количество потоков, 300 – количество итераций, 3 – размер медианного фильтра.

./lab2 input.txt output.txt 4 300 3

Перейдём в другое окно терминала и выполним следующие команды, чтобы проверить количество выделенных потоков:

top

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
16613	coder	20	0	66812	33304	1916	S	420.0	0.2	0:21.28	lab2

ps -o pid,nlwp 16613

PID	NLWP
16613	5

Из вывода в терминал видно, что программа задействовала 5 потоков, из них 1 – это главный поток, а остальные 4 – потоки, выделенные для вычислений.

Исследование ускорения и эффективности

Условия эксперимента: размер матрицы 2000 на 2000, число итераций K = 3, размер окна 3×3. Время замерялось внутри программы (clock_gettime, CLOCK_MONOTONIC) и включает только фильтрацию, без ввода-вывода. Программа собрана с флагом -pthread. Для каждого числа потоков выполнено по три запуска, в таблице приведено среднее арифметическое. Ускорение и эффективность считались по формулам:

$$S = \frac{T_s}{T_p}$$

где T_s – время последовательной реализации, T_p – время параллельной реализации, p – количество ядер.

$$X = \frac{S}{p}$$

где S – ускорение.

В эксперименте $T_s = 2954$ мс, $p = 6$. Замеры для каждого значения k проводились по 3 раза, после чего в таблицу заносилось среднее значение.

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
1	2954	1	1
2	1511	1.955	0.977
3	1026	2.878	0.960
4	769	3.840	0.959
5	639	4.617	0.923
6	632	4.670	0.778
16	569	5.186	0.324
128	559	5.277	0.041
1024	571	5.174	0.005

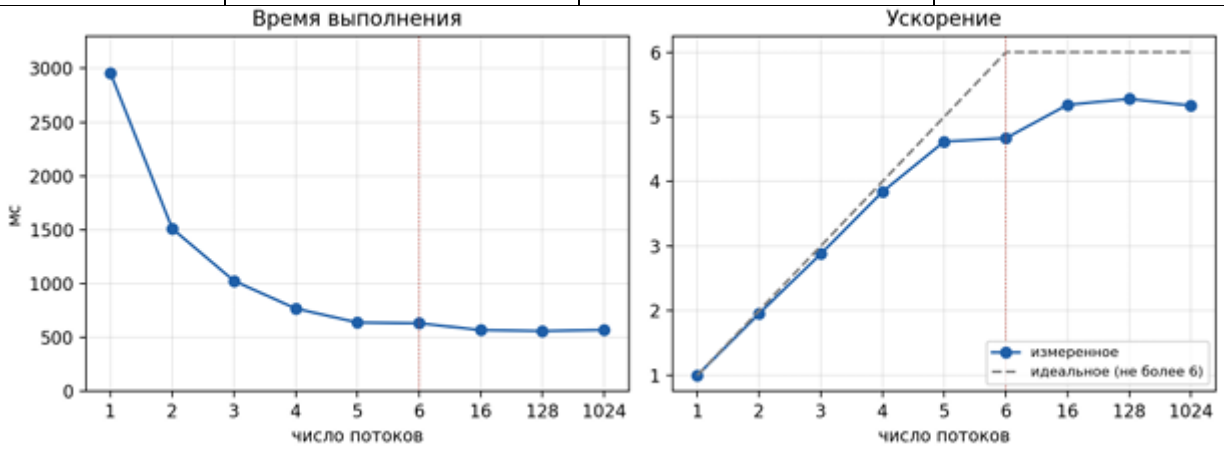


Рис. 1. Время выполнения и ускорение (пунктир – граница в 6 ядер)

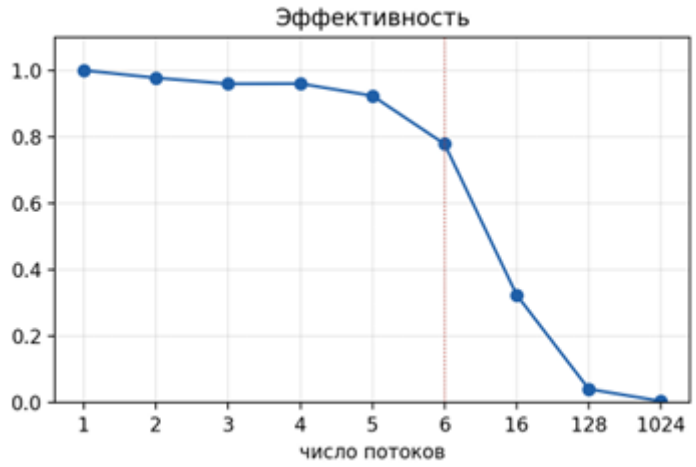


Рис. 2. Эффективность

В результате измерений были выяснены зависимости исследуемых величин от задействованного количества потоков:

- До 5 потоков ускорение почти линейное. Строки обрабатываются независимо, потоки не делят изменяемых данных, а синхронизация сводится к одному барьеру на итерацию, поэтому накладные расходы малы.
- На 5-6 потоках (по числу ядер) ускорение составило 4.67 при эффективности 0.78, а разброс между запусками наибольший (544–683 мс). Шесть рабочих потоков занимают все ядра, и любая фоновая нагрузка задерживает один из потоков. Полосы строк распределены между потоками статически, а после каждой итерации все ждут на барьере самого медленного, поэтому одна задержка замедляет всю итерацию. Лучший запуск (544 мс) даёт ускорение 5.43, что близко к идеалу. Кроме того, при полной загрузке всех ядер процессор обычно работает на меньшей частоте, чем при одном потоке.
- Больше 6 потоков дают небольшое ускорение времени выполнения за счёт измельчения полос строк, обрабатываемых одним процессом. планировщик ОС может динамически распределять потоки по ядрам: ядро, освободившееся раньше, берёт другой поток. Это частично компенсирует задержки отдельных ядер, которые при ровно 6 потоках приводят к простою остальных на барьере.
- При 1024 потоках время слегка растёт (571 мс): создание и уничтожение потоков, память под стеки и окна, переключения контекста и вытеснение данных из кэша становятся заметны, а выигрыша в балансировке уже нет.

Эффективность при $p > 6$ начинает падать, поскольку ускорение остаётся почти постоянным, а число потоков растёт.

Вывод

В ходе работы реализовано многопоточное наложение медианного фильтра на целочисленную матрицу K раз с настраиваемым размером окна и числом потоков, задаваемыми аргументами командной строки. Потоки создаются средствами POSIX Threads, обрабатывают непересекающиеся полосы строк и синхронизируются барьером после каждой итерации; для двойной буферизации результат выбирается по чётности числа итераций. Результаты многопоточного и однопоточного режимов совпадают. Исследование на 6-ядерном процессоре показало почти линейный рост ускорения до 5 потоков (эффективность 0.92–0.98) и выход на плато около 5.2–5.3 при 6 и более потоках, так как число одновременно работающих потоков ограничено числом ядер. Падение эффективности при большом числе потоков объясняется неизменным ускорением при росте p, а повышенный разброс времени на 6 потоках – статическим делением строк и ожиданием самого медленного потока на барьере. Для практического применения число потоков стоит выбирать равным числу ядер процессора.